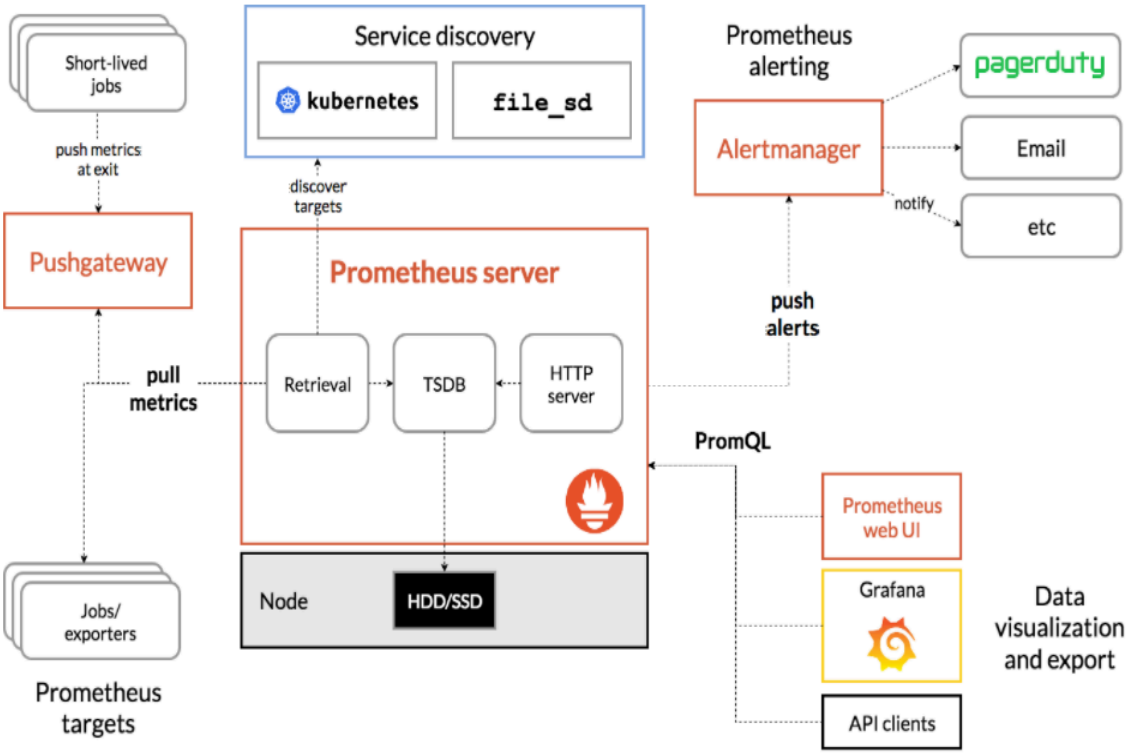
Setup Prometheus Monitoring On Kubernetes Cluster

Prometheus is a high-scalable open-source monitoring framework. It provides out-of-the-box monitoring capabilities for the Kubernetes container orchestration platform. Also, In the observability space, it is gaining huge popularity.

high-level architecture of Prometheus :



Tasks:

1. Setup Prometheus (kubernetes)

2. Prometheus Deployment

3. Connecting To Prometheus Dashboard

4. Exposing Prometheus Using Ingress

5. Setting Up Kube State Metrics

6. Setup Prometheus Node Exporter

STEP 1 :

Setup Prometheus (kubernetes)

Node : master-node

we will create a Kubernetes namespace for all our monitoring components :

Create a Namespace :

kubectl create namespace monitoring

create a directory :

mkdir prometheus-kube

cd prometheus-kube

Create a ClusterRole :

Create a file named clusterRole.yaml :

sudo nano clusterRole.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus
rules:
- apiGroups: [""]
  resources:
  - nodes
  - nodes/proxy
  - services
  - endpoints
  - pods
  verbs: ["get", "list", "watch"]
- apiGroups:
  - extensions
  resources:
  - ingresses
  verbs: ["get", "list", "watch"]
- nonResourceURLs: ["/metrics"]
  verbs: ["get"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus
roleRef:
  apiGroup: rbac.authorization.k8s.io
```

```
  kind: ClusterRole
  name: prometheus
subjects:
- kind: ServiceAccount
  name: default
  namespace: monitoring
```

kubectl apply -f clusterRole.yaml

Create a file config-map.yaml :

sudo nano config-map.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-server-conf
  labels:
    name: prometheus-server-conf
  namespace: monitoring
data:
  prometheus.rules: |-
    groups:
    - name: devopscube demo alert
      rules:
      - alert: High Pod Memory
        expr: sum(container_memory_usage_bytes) > 1
        for: 1m
        labels:
          severity: slack
        annotations:
          summary: High Memory Usage
  prometheus.yml: |-
    global:
      scrape_interval: 5s
      evaluation_interval: 5s
    rule_files:
      - /etc/prometheus/prometheus.rules
    alerting:
      alertmanagers:
      - scheme: http
        static_configs:
        - targets:
          - "alertmanager.monitoring.svc:9093"

    scrape_configs:
      - job_name: 'node-exporter'
        kubernetes_sd_configs:
          - role: endpoints
        relabel_configs:
        - source_labels: [__meta_kubernetes_endpoints_name]
          regex: 'node-exporter'
          action: keep

      - job_name: 'kubernetes-apiservers'

        kubernetes_sd_configs:
        - role: endpoints
        scheme: https

        tls_config:
          ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
        bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token

        relabel_configs:
```

```
        - source_labels: [__meta_kubernetes_namespace, __meta_kubernetes_service_name,
__meta_kubernetes_endpoint_port_name]
          action: keep
          regex: default;kubernetes;https

      - job_name: 'kubernetes-nodes'

        scheme: https

        tls_config:
          ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
        bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token

        kubernetes_sd_configs:
        - role: node

        relabel_configs:
        - action: labelmap
          regex: __meta_kubernetes_node_label_(.+)
        - target_label: __address__
          replacement: kubernetes.default.svc:443
        - source_labels: [__meta_kubernetes_node_name]
          regex: (.+)
          target_label: __metrics_path__
          replacement: /api/v1/nodes/${1}/proxy/metrics

      - job_name: 'kubernetes-pods'

        kubernetes_sd_configs:
        - role: pod

        relabel_configs:
        - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
          action: keep
          regex: true
        - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
          action: replace
          target_label: __metrics_path__
          regex: (.+)
        - source_labels: [__address__, __meta_kubernetes_pod_annotation_prometheus_io_port]
          action: replace
          regex: ([^:]+)(?::\d+)?;(\d+)
          replacement: $1:$2
          target_label: __address__
        - action: labelmap
          regex: __meta_kubernetes_pod_label_(.+)
        - source_labels: [__meta_kubernetes_namespace]
          action: replace
          target_label: kubernetes_namespace
        - source_labels: [__meta_kubernetes_pod_name]
          action: replace
          target_label: kubernetes_pod_name

      - job_name: 'kube-state-metrics'
        static_configs:
          - targets: ['kube-state-metrics.kube-system.svc.cluster.local:8080']

      - job_name: 'kubernetes-cadvisor'

        scheme: https

        tls_config:
          ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
        bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token

        kubernetes_sd_configs:
        - role: node

        relabel_configs:
        - action: labelmap
          regex: __meta_kubernetes_node_label_(.+)
        - target_label: __address__
          replacement: kubernetes.default.svc:443
        - source_labels: [__meta_kubernetes_node_name]
          regex: (.+)
          target_label: __metrics_path__
          replacement: /api/v1/nodes/${1}/proxy/metrics/cadvisor
```

```
      - job_name: 'kubernetes-service-endpoints'

        kubernetes_sd_configs:
        - role: endpoints

        relabel_configs:
        - source_labels: [__meta_kubernetes_service_annotation_prometheus_io_scrape]
          action: keep
          regex: true
        - source_labels: [__meta_kubernetes_service_annotation_prometheus_io_scheme]
          action: replace
          target_label: __scheme__
          regex: (https?)
        - source_labels: [__meta_kubernetes_service_annotation_prometheus_io_path]
          action: replace
          target_label: __metrics_path__
          regex: (.+)
        - source_labels: [__address__,
__meta_kubernetes_service_annotation_prometheus_io_port]
          action: replace
          target_label: __address__
          regex: ([^:]+)(?::\d+)?;(\d+)
          replacement: $1:$2
        - action: labelmap
          regex: __meta_kubernetes_service_label_(.+)
        - source_labels: [__meta_kubernetes_namespace]
          action: replace
          target_label: kubernetes_namespace
        - source_labels: [__meta_kubernetes_service_name]
          action: replace
          target_label: kubernetes_name
```

kubectl apply -f config-map.yaml

STEP 2 :

Prometheus Deployment

Create a file named prometheus-deployment.yaml:

sudo nano prometheus-deployment.yaml

Add this configuration to the file:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prometheus-deployment
  namespace: monitoring
  labels:
    app: prometheus-server
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus-server
  template:
    metadata:
      labels:
        app: prometheus-server
    spec:
      containers:
        - name: prometheus
          image: prom/prometheus
          args:
            - "--storage.tsdb.retention.time=12h"
            - "--config.file=/etc/prometheus/prometheus.yml"
            - "--storage.tsdb.path=/prometheus/"
          ports:
            - containerPort: 9090
          resources:
            requests:
```

```
          cpu: 500m
          memory: 500M
        limits:
          cpu: 1
          memory: 1Gi
      volumeMounts:
        - name: prometheus-config-volume
          mountPath: /etc/prometheus/
        - name: prometheus-storage-volume
          mountPath: /prometheus/
  volumes:
    - name: prometheus-config-volume
      configMap:
        defaultMode: 420
        name: prometheus-server-conf

    - name: prometheus-storage-volume
      emptyDir: {}
```

kubectl apply -f prometheus-deployment.yaml

check the created deployment :

kubectl get deployments --namespace=monitoring

kubectl get pods -n monitoring

kubectl describe pod prometheus-deployment-b65d5d898-ntdz9 -n monitoring

Almost 10 minutes: Please wait a while for the pod to be created*

STEP 3 :

Connecting To Prometheus Dashboard

Exposing Prometheus as a Service NodePort

expose Prometheus on all kubernetes node IP on port 30000

Create a file named prometheus-service.yaml :

sudo nano prometheus-service.yaml

Add this configuration to the file:

```
apiVersion: v1
kind: Service
metadata:
  name: prometheus-service
  namespace: monitoring
  annotations:
      prometheus.io/scrape: 'true'
      prometheus.io/port:   '9090'
spec:
  selector:
    app: prometheus-server
  type: NodePort
  ports:
    - port: 8080
      targetPort: 9090
      nodePort: 30000
```

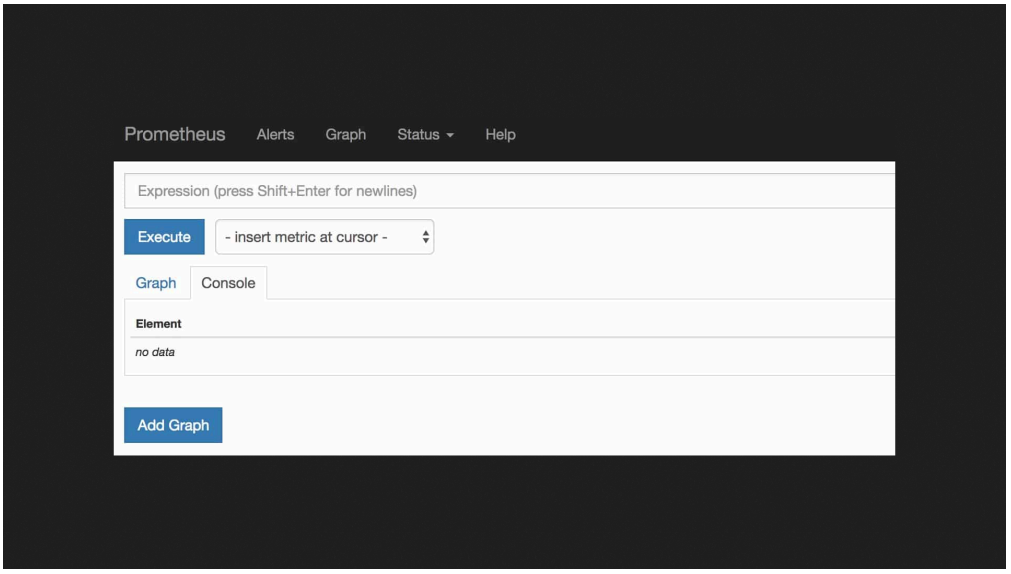
kubectl apply -f prometheus-service.yaml --namespace=monitoring

you can access the Prometheus dashboard using any of the Kubernetes nodes IP on port 30000

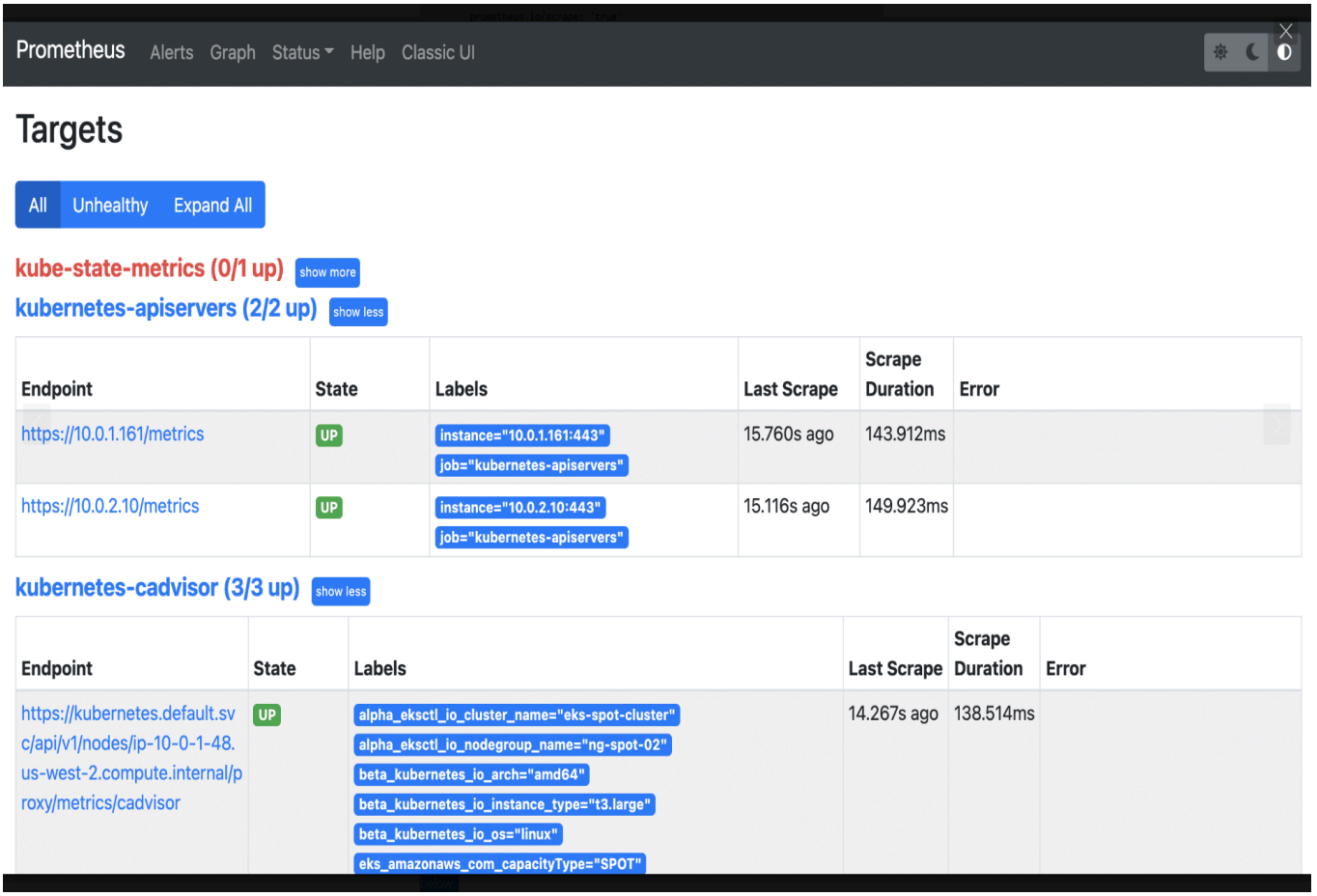
example :

http://IP-master-node:30000

http://x.x.x.x:30000



If you browse to status --> Targets, you will see all the Kubernetes endpoints connected to Prometheus automatically using service discovery as shown below.



STEP 4 :

Exposing Prometheus Using Ingress :

Check ingress controller :

kubectl get pods -n ingress-nginx

The container (nginx-controller) must be in the Running status

Create a file named prometheus-ingress.yaml :

sudo nano prometheus-ingress.yaml

Add this configuration to the file :

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: prometheus-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: prometheus-test.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: prometheus-service
                port:
                  number: 8080
```

kubectl apply -f prometheus-ingress.yaml --namespace=monitoring

Check ingress :

kubectl get ingress -n monitoring

Edit the /etc/hosts file using the below command in Terminal:

sudo nano /etc/hosts

x.x.x.x prometheus-test.local

Now save and close the /etc/hosts file.

you can access the Prometheus dashboard with

http://prometheus-test.local

STEP 5 :

Setting Up Kube State Metrics

Kube State metrics is a service that talks to the Kubernetes API server to get all the details about all the API objects like deployments, pods, daemonsets, Statefulsets, etc.

Primarily it produces metrics in Prometheus format with the stability as the Kubernetes API. Overall it provides kubernetes objects & resources metrics that you cannot get directly from native Kubernetes monitoring components.

Kube state metrics service exposes all the metrics on /metrics URI. Prometheus can scrape all the metrics exposed by Kube state metrics.

Following are some of the important metrics you can get from Kube state metrics :

1. Node status, node capacity (CPU and memory)
2. Replica-set compliance (desired/available/unavailable/updated status of replicas per deployment)
3. Pod status (waiting, running, ready, etc)
4. Ingress metrics
5. PV, PVC metrics
6. Daemonset & Statefulset metrics.
7. Resource requests and limits.
8. Job & Cron Job metrics

Kube State Metrics Setup :

copy manifest files in master-node using winscp

Create all the objects by pointing to the manifest directory :

kubectl apply -f kube-state-metrics-configs/

Check the deployment status :

kubectl get deployments kube-state-metrics -n kube-system

STEP 6 :

Setup Prometheus Node Exporter

By default, most of the Kubernetes clusters expose the metric server metrics (Cluster level metrics from the summary API) and Cadvisor (Container level metrics). It does not provide detailed node-level metrics.

To get all the kubernetes node-level system metrics, you need to have a node-exporter running in all the kubernetes nodes. It collects all the Linux system metrics and exposes them via /metrics endpoint on port 9100

Deploy node exporter on all the Kubernetes nodes as a daemonset.

Create a directory :

mkdir node-exporter

cd node-exporter

Create a file name daemonset.yaml:

sudo nano daemonset.yaml

Add this configuration to the file :

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  labels:
    app.kubernetes.io/component: exporter
    app.kubernetes.io/name: node-exporter
  name: node-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app.kubernetes.io/component: exporter
      app.kubernetes.io/name: node-exporter
  template:
    metadata:
      labels:
        app.kubernetes.io/component: exporter
        app.kubernetes.io/name: node-exporter
    spec:
```

```
    containers:
    - args:
      - --path.sysfs=/host/sys
      - --path.rootfs=/host/root
      - --no-collector.wifi
      - --no-collector.hwmon
      -
--collector.filesystem.ignored-mount-points=^/(dev|proc|sys|var/lib/docker/.+|var/lib/kubelet/pods/.+)($|/)
      - --collector.netclass.ignored-devices=^(veth.*)$
      name: node-exporter
      image: prom/node-exporter
      ports:
        - containerPort: 9100
          protocol: TCP
      resources:
        limits:
          cpu: 250m
          memory: 180Mi
        requests:
          cpu: 102m
          memory: 180Mi
      volumeMounts:
      - mountPath: /host/sys
        mountPropagation: HostToContainer
        name: sys
        readOnly: true
      - mountPath: /host/root
        mountPropagation: HostToContainer
        name: root
        readOnly: true
    volumes:
    - hostPath:
        path: /sys
      name: sys
    - hostPath:
        path: /
      name: root
```

kubectl apply -f daemonset.yaml

Check the deployment status :

kubectl get daemonset -n monitoring

Create a file name service.yaml :

sudo nano service.yaml

Add this configuration to the file :

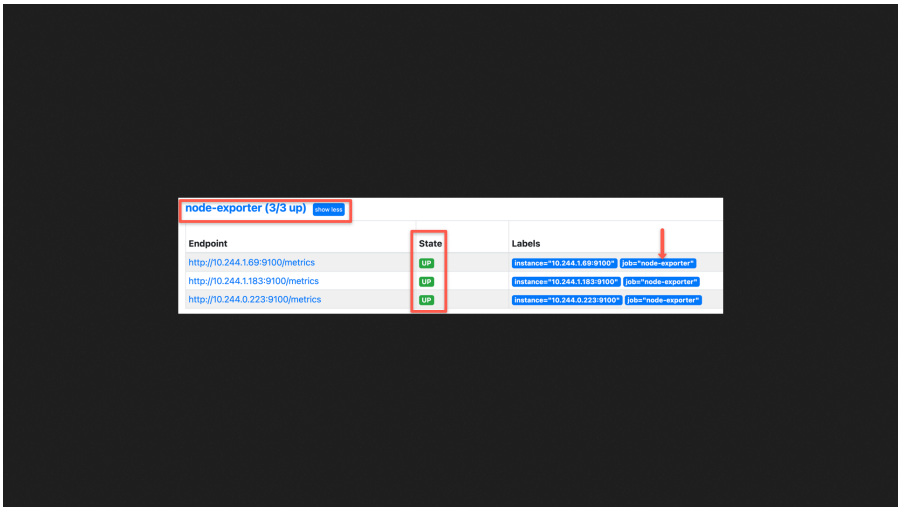
```
apiVersion: v1

kind: Service

metadata:
  name: node-exporter
  namespace: monitoring
  annotations:
     prometheus.io/scrape: 'true'
     prometheus.io/port:   '9100'
spec:
  selector:
     app.kubernetes.io/component: exporter
     app.kubernetes.io/name: node-exporter
  ports:
  - name: node-exporter
    protocol: TCP
    port: 9100
    targetPort: 9100
```

kubectl apply -f service.yaml

Check the service status :

kubectl get endpoints -n monitoring

If you browse to status --> Targets, you will see all the Kubernetes endpoints connected to Prometheus automatically using service discovery as shown below.



Successful

END